

01. 關於行動應用程式 (APP)

一、認識 APP



行動應用程式 (簡稱App) 是一種電腦程式或軟體應用，設計用於在手機、平板電腦或智慧手錶等行動裝置上運作。行動應用程式通常與桌面應用程式 (設計用於在桌上型電腦上運行) 以及 Web 應用程式 (在行動網路瀏覽器中運行，而不是直接在行動裝置上運行) 比較。

應用程式最初旨在提高工作效率，例如電子郵件、日曆和聯絡人資料庫，但隨著對應用程式的需求促使其迅速擴展到其他領域，例如手機遊戲、工廠自動化、GPS 位置的服務、訂單追蹤和購票，因此現在有數百萬個應用程式可供使用。許多應用程式需要連接網路，。應用程式通常從應用程式商店下載。

「app」一詞是「application」(應用程式) 的縮寫，此後變得非常流行；2010 年，它被美國方言協會評為「年度詞彙」。

應用程式大致可分為三種類型：原生應用程式、Web 應用程式和混合應用程式。原生應用程式專為行動作業系統 (通常是 iOS 或 Android) 設計。Web 應用程式使用HTML5或CSS編寫，通常透過瀏覽器運行。混合應用程式使用JavaScript、CSS 和 HTML5等 Web 技術構建，其功能類似於 Web 應用程序，只是封裝在原生容器中。

(一) 原生應用程式

原生應用程式專門針對單一平台 (例如 iOS 或 Android) 而生。這些應用程式是使用平台偏好的程式語言和工具所建立，以便充分利用內建的功能。針對 iOS 應用程式，最常見的程式語言是 Swift 或 Objective-C，而 Android 開發人員通常使用 Java 或 Kotlin。



(二) Web 應用程式

裝置上不會安裝網路應用程式，但可透過網頁瀏覽器存取。網路應用程式基本上外觀和風格類似於行動應用程式，但能夠在具備網際網路存取的裝置上運作。網路應用程式具備觸及性和相容性，但可能不會提供原生應用程式的所有功能。

(三) 混合應用程式

混合式應用程式結合原生應用程式和網路應用程式的要素。這些混合式應用程式是使用 HTML、CSS 和 JavaScript 等網路技術所建立，但會包裝在原生應用程式殼層中。這表示可在裝置上安裝這些混合式應用程式，並可存取裝置內建的部分功能。混合式應用程式是一種良好的折衷方案，在效能和成本效益之間取得平衡。

二、跨平台應用程式開發

在企業數位轉型日益加速的現今，手機 APP 已經不是僅僅作為附加工具，而是品牌、流程、使用者體驗與營運效率的核心節點。無論你是想為客戶提供更貼近的行動服務、還是希望將內部流程移轉至行動裝置，APP 開發的決策早已不再屬於技術團隊，而是經營階層、產品管理部門與 IT 決策者共同參與的戰略議題。然而，對許多企業而言，「選擇哪種開發方式」仍舊是一個困難的問題。傳統選項為 iOS 與 Android 分別開發、或採用混合式技術。但在追求速度、成本、維運與跨平台一致性時，許多企業正在將目光轉向跨平台應用程式開發。

(一) React Native (<https://reactnative.dev/>)

React Native 是一種由 Facebook 開發並維護的開源移動應用框架，旨在讓開發者能夠使用 JavaScript 和 React 來建造具有原生表現力的 iOS 和 Android 應用。自 2015 年推出以來，它已經快速成長為最受歡迎的跨平台移動開發解決方案之一。此框架的核心理念在於「學習一次，隨處撰寫」(Learn Once, Write Anywhere)，意即開發者無需分別學習不同平台特定語言 (如 Objective-C/Swift 或 Java/Kotlin)，就可以創建涵蓋多個平台的應用程序。

雖然 React Native 擁有許多優點，但在開發上還是有一些限制在，像是若要使用 React 官方沒有提供的功能時，需尋找適合的第三方套件，若第三方套件也沒有適合時，就只能自己撰寫開發原生模組，再透過 React Native 橋接的方式讓應用程式使用，因此必要時開發者仍須具備原生 APP 的撰寫能力。不過這項缺點在 React Native 社群蓬勃發展後已大幅降低，在社群中有著大量的第三方套件資源，非常容易找到適合的套件。

(二) Flutter (<https://flutter.dev/>)

Flutter 是一個由 Google 創建的開源框架，使用的語言稱為 Dart。它是一個針對 Web 平臺、行行動和桌面應用程式的軟體開發工具，使用的語言是 Dart。儘管 Dart 不像 JavaScript 那麼受歡迎，但它仍然是一種相對容易掌握的良好編程語言。通常有 Java 和 C++ 背景的開發者比 JavaScript 開發者更容易理解 Dart。Flutter 也被視為增強型 UI 工具，可用於僅使用單一代碼庫開發跨平臺應用程式。它提供了靈活和表達豐富的原生性能 UI 給 Flutter 開發者使用，由 Google 開發團隊以及整個 Flutter 社群提供支援。



比較項目	Native APP	Flutter 	React Native	Hybrid APP	Web APP
效能	 高 	高 (原生編譯)	接近高 	良好 	低 
成本	高	中	中	較低	低
網路要求	無要求	無要求	無要求	無要求	依賴網路
跨平台	無	有 (高度一致)	有	有	有(瀏覽器)
開發語言	Java Objective-C Swift	 Dart	JavaScript	HTML CSS JavaScript	HTML CSS JavaScript
開發難度	難	中等 (須學習 Dart 語言與 Flutter 框架)	較簡單 (須了解 ES6)	簡單	簡單
APP 發佈	APP Store Google Play	APP Store Google Play	APP Store Google Play	APP Store Google Play	Web Server
適合應用	大型 APP、 遊戲 	中大型 APP、遊 戲、精美 UI APP 	中型 APP	中型 APP	小型 APP

三、行動應用程式開發階段



(一) 構思與市場研究

行動應用程式開發的第一步是定義應用構想並進行全面市場研究。此階段包含：鎖定目標受眾、分析競爭對手缺口與機會，以及驗證應用的獨特價值主張。理解用戶痛點與市場需求有助企業打造解決現實問題的產品。無論目標是提升銷售、改善客戶服務體驗，或提高組織運作效率，皆可運用 Google Trends、App Store 分析及競爭者標竿比較等工具精煉概念。

在此階段結束時，應明確掌握三大要素：應用核心目的、目標用戶輪廓及盈利模式（若適用）。

(二) 定義商業與技術需求

一旦應用構想驗證完成，下一步即需文件化商業與技術需求。此階段包含定義核心功能、用戶流程、技術堆疊與開發方法。根據預算與用戶需求，必須決定開發原生應用（專為特定平台如 iOS 或 Android 運行的應用程式，可完全掌控系統資源並提供更高性能），或跨平台應用（重複使用相同程式碼於多平台，縮短開發時程並減少錯誤）。一份結構完善的行動應用開發藍圖還需規劃：第三方服務整合、API 串接、後端基礎架構及安全協定，以支援擴展性與效能優化。

(三) 建立線框圖與原型

在實際開發啟動前，設計師需建立線框圖與原型，將行動應用的使用者介面 (UI) 和使用者體驗 (UX) 可視化。線框圖有助於規劃應用結構、導航流程及互動邏輯，確保設計直覺且用戶友好。透過 Figma 或 Adobe XD 等工具，團隊可建立互動式原型，並邀請潛在用戶進行測試。

此階段關鍵在於收集回饋以優化設計，降低開發期間發生重大UI/UX變更的風險。完善的線框圖能確保開發流程順暢推進，強化用戶參與度與留存率。

(四) 最小可行產品 (MVP) 開發

最小可行產品 (MVP) 是僅包含核心功能的應用基礎版本。其主要目的是測試市場需求，故此階段的目標是以最低資源快速開發並發布，收集用戶回饋，並基於真實使用數據迭代優化。

Spotify 即為成功 MVP 的典範案例：

這個全球最大音樂串流服務初始目標是解決音樂盜版問題。創辦團隊未直接發布完整產品，而是開發僅限桌機版的MVP——具備有限音樂庫與邀請制機制，僅耗時四個月便完成驗證市場需求。關鍵創新在於透過閃電級串流技術實現即時無緩衝播放體驗，使Spotify MVP早期即獲成功，吸引早期採用者與唱片公司協助精進產品，逐步新增個人化功能後才全球擴張。

注意：MVP應視為最終應用的基石，協助逐步完善功能架構。

(五) 全面開發與測試

最小可行產品 (MVP) 驗證完成後，下一步即進入全面開發階段。此階段包含後端與前端開發，確保所有功能皆依行動應用藍圖建置。Scrum 與 Kanban 等敏捷方法透過將開發拆分為衝刺 (sprint) 與里程碑，協助團隊高效管理任務。

測試對確保應用的功能、安全性與效能至關重要。多元測試方法包含：單元測試、功能測試、效能測試及公測，有助於正式上線前識別並修復錯誤。透過 TestFlight (iOS) 及 Google Play Beta (Android) 等工具，真實用戶可測試應用並提供寶貴回饋。嚴謹的行動應用測試策略能提升應用效能、用戶滿意度及留存率。

(六) 部署與上線策略

成功的應用上線需要周詳規劃與執行。於 Google Play Store與 Apple App Store 發布應用時，需遵循應用提交規範、優化中繼資料以提升應用商店能見度 (ASO)、準備行銷素材 (截圖、宣傳影片，若能提供用戶推薦更佳)。

此外，上市策略還應包括預發布活動、網紅合作、付費廣告和社群媒體推廣。務必設置必要的分析工具並追蹤關鍵指標，如下載量、留存率和用戶參與度，以識別需要改進的領域。

最後，請確保與用戶群保持互動。傾聽並回應他們關於錯誤、更新和功能建議的反饋，這將有助於建立忠誠度，並在競爭激烈的市場中脫穎而出。